



基于新一代 Kaldi 的语音识别

匡方军

2025.10.13



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

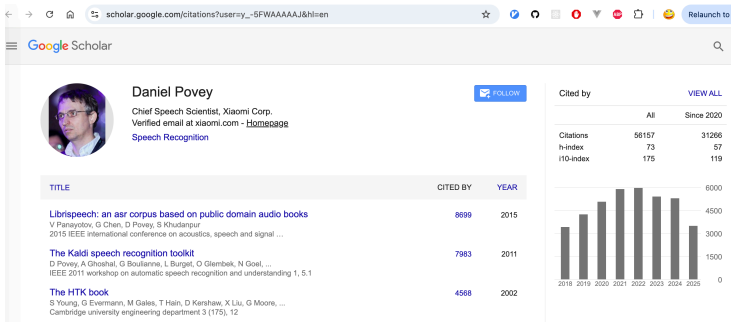
- Transducer

新一代 Kaldi 实战

总结

Kaldi 之父 (1/2)

- ▶ Daniel Povey 博士毕业于英国剑桥大学
- ▶ 2019 年 10 月加入小米，担任集团语音首席科学家
- ▶ 2022 年 12 月，入选 **IEEE Fellow**

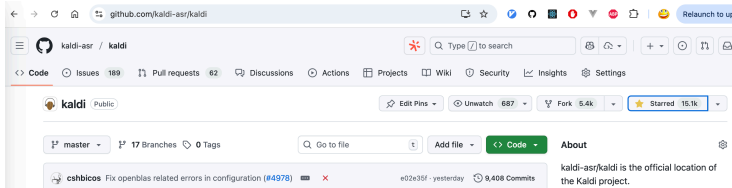


Kaldi 之父 (2/2)



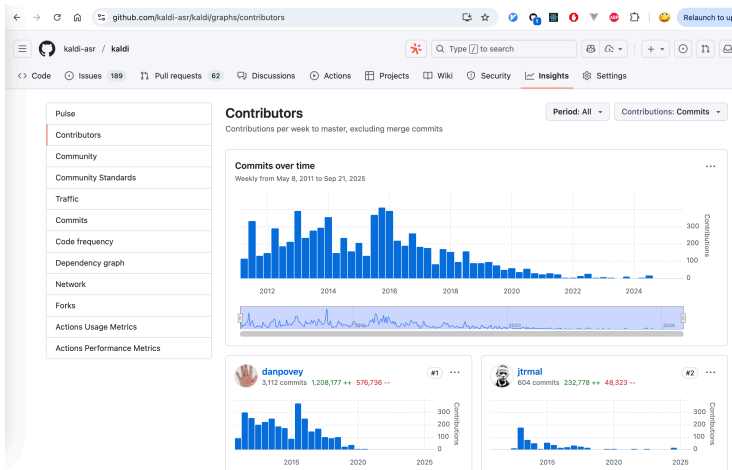
Kaldi

► <https://github.com/kaldi-asr/kaldi>



Kaldi

► <https://github.com/kaldi-asr/kaldi>



新一代 Kaldi 团队



新一代 Kaldi 社区

- ▶ 我们有**非常活跃**的**社区**



新一代 Kaldi 微信公众号



微信交流 9 群



团队成员微信



QQ 交流群 (群号: 744602236)

- ▶ 微信群超过 200 人时，需要邀请进群

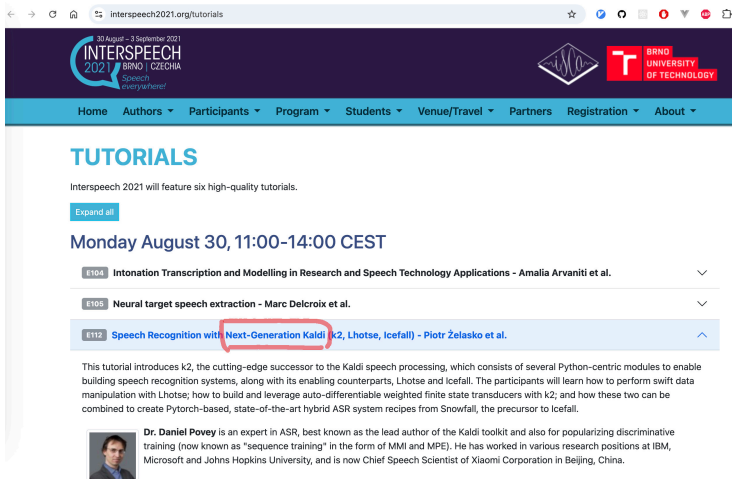
新一代 Kaldi 项目

- ▶ 核心算法 (**k2**)
 - ▶ <https://github.com/k2-fsa/k2>
 - ▶ 含义: kaldi2, 世界第二高峰
 - ▶ 基于 C++ 和 CUDA
- ▶ 数据处理 (**lhotse**)
 - ▶ <https://github.com/lhotse-speech/lhotse>
 - ▶ 含义: 世界第四高峰
- ▶ 训练 (**icefall**)
 - ▶ <https://github.com/k2-fsa/icefall>
 - ▶ 基于 PyTorch
- ▶ 部署 (**sherpa**)
 - ▶ 基于 **torchscript**: <https://github.com/k2-fsa/sherpa>
 - ▶ 基于 **onnxruntime**: <https://github.com/k2-fsa/sherpa-onnx>
 - ▶ 基于 **ncnn**: <https://github.com/k2-fsa/sherpa-ncnn>
 - ▶ 含义: 夏尔巴人, 专业的登山向导

新一代 Kaldi 论文

- Zhu H, et al. ZipVoice-Dialog: Non-Autoregressive Spoken Dialogue Generation with Flow Matching. arXiv preprint arXiv:2507.09318, 2025.
- Zhu H, et al. ZipVoice: Fast and High-Quality Zero-Shot Text-to-Speech with Flow Matching. arXiv preprint arXiv:2506.13053, 2025. **IEEE ASRU 2025**
- Yang Y, et al. k2SSL: A faster and better framework for self-supervised speech representation learning. arXiv preprint arXiv:2411.17100, 2024. **ICME 2025**
- Yao Z, et al. **CR-CTC**: Consistency regularization on CTC for improved speech recognition[J]. arXiv preprint arXiv:2410.05101, 2024. **ICLR 2025**
- Jin Z, et al. LibriheavyMix: a 20,000-hour dataset for single-channel reverberant multi-talker speech separation, ASR and speaker diarization[J]. arXiv preprint arXiv:2409.00819, 2024. **INTERSPEECH 2024**
- Kang W, et al. Libriheavy: A 50,000 hours ASR corpus with punctuation casing and context[C]//ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2024: 10991-10995. **ICASSP 2024**
- Yang, Xiaoyu, et al. "PromptASR for contextualized ASR with controllable style." ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2024. **ICASSP 2024**
- Yao, Zengwei, et al. "**Zipformer**: A faster and better encoder for automatic speech recognition." arXiv preprint arXiv:2310.11230 (2023). **ICLR 2024 (Oral)**
- Kang, Wei, et al. "Delay-penalized transducer for low-latency streaming ASR." ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2023. **ICASSP 2023**
- Kang, Wei, et al. "Fast and parallel decoding for transducer." ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2023. **ICASSP 2023**
- Guo, Liyong, et al. "Predicting multi-codebook vector quantization indexes for knowledge distillation." ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2023. **ICASSP 2023**
- Yang, Yifan, et al. "Blank-regularized ctc for frame skipping in neural transducer." arXiv preprint arXiv:2305.11558 (2023). **INTERSPEECH 2023**
- Yao, Zengwei, et al. "Delay-penalized CTC implemented based on Finite State Transducer." arXiv preprint arXiv:2305.11539 (2023). **INTERSPEECH 2023**
- Kuang, Fangjun, et al. "Pruned RNN-T for fast, memory-efficient ASR training." arXiv preprint arXiv:2206.13236 (2022). **INTERSPEECH 2022**

新一代 Kaldi Interspeech 2021 tutorial



The screenshot shows the Interspeech 2021 website. The header includes the event logo (30 August - 3 September 2021, INTERSPEECH 2021, BRNO | CZECHIA, Speech everywhere!), the MITC logo, and the Brno University of Technology logo. A navigation bar lists: Home, Authors, Participants, Program, Students, Venue/Travel, Partners, Registration, and About. The main section is titled "TUTORIALS" and states that Interspeech 2021 will feature six high-quality tutorials. An "Expand all" button is present. A list of tutorials is shown, with the third one highlighted in blue and its title "Speech Recognition with Next-Generation Kaldi (k2, Lhotse, Icefall)" circled in red. The tutorial is by Piotr Żelasko et al. Below the list, a paragraph describes the k2 framework as a cutting-edge successor to Kaldi, built with Python-centric modules for building speech recognition systems, leveraging Lhotse for data manipulation and k2 for auto-differentiable weighted finite state transducers. A small portrait of Dr. Daniel Povey is shown next to his bio, which identifies him as an expert in ASR, lead author of the Kaldi toolkit, and Chief Speech Scientist at Xiaomi Corporation.

30 August - 3 September 2021
INTER SPEECH
 2021
 BRNO | CZECHIA
 Speech everywhere!

MITC
 BRNO UNIVERSITY OF TECHNOLOGY

Home Authors Participants Program Students Venue/Travel Partners Registration About

TUTORIALS

Interspeech 2021 will feature six high-quality tutorials.

[Expand all](#)

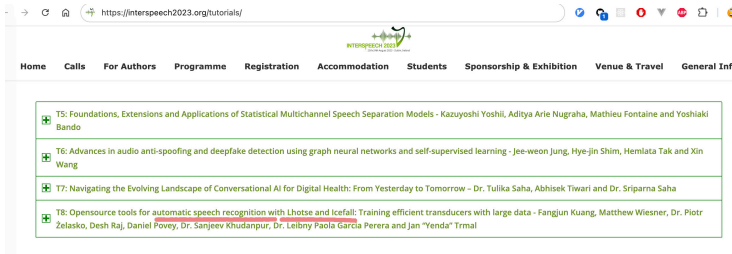
Monday August 30, 11:00-14:00 CEST

- E104 Intonation Transcription and Modelling in Research and Speech Technology Applications - Amalia Arvaniti et al.
- E105 Neural target speech extraction - Marc Delcroix et al.
- E112 Speech Recognition with Next-Generation Kaldi (k2, Lhotse, Icefall) - Piotr Żelasko et al.**

This tutorial introduces k2, the cutting-edge successor to the Kaldi speech processing, which consists of several Python-centric modules to enable building speech recognition systems, along with its enabling counterparts, Lhotse and Icefall. The participants will learn how to perform swift data manipulation with Lhotse; how to build and leverage auto-differentiable weighted finite state transducers with k2; and how these two can be combined to create Pytorch-based, state-of-the-art hybrid ASR system recipes from Snowfall, the precursor to Icefall.

 **Dr. Daniel Povey** is an expert in ASR, best known as the lead author of the Kaldi toolkit and also for popularizing discriminative training (now known as "sequence training" in the form of MMI and MPE). He has worked in various research positions at IBM, Microsoft and Johns Hopkins University, and is now Chief Speech Scientist of Xiaomi Corporation in Beijing, China.

新一代 Kaldi Interspeech 2023 tutorial

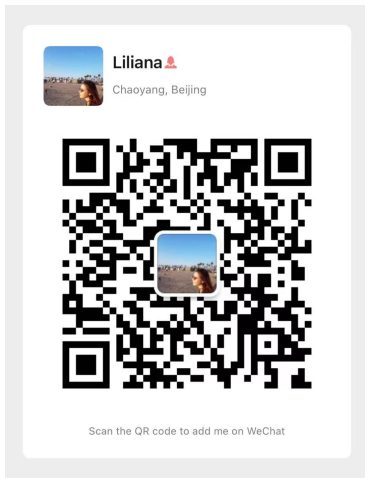


The screenshot shows the Interspeech 2023 website. The browser address bar displays <https://interspeech2023.org/tutorials/>. The website has a navigation menu with the following items: Home, Calls, For Authors, Programme, Registration, Accommodation, Students, Sponsorship & Exhibition, Venue & Travel, and General Inf. Below the menu, there is a list of tutorials, each with a small icon and a title:

- T5: Foundations, Extensions and Applications of Statistical Multichannel Speech Separation Models - Kazuyoshi Yoshii, Aditya Arie Nugraha, Mathieu Fontaine and Yoshiaki Bando
- T6: Advances in audio anti-spoofing and deepfake detection using graph neural networks and self-supervised learning - Jee-weon Jung, Hye-jin Shim, Hemlata Tak and Xin Wang
- T7: Navigating the Evolving Landscape of Conversational AI for Digital Health: From Yesterday to Tomorrow - Dr. Tulika Saha, Abhisek Tiwari and Dr. Sriparna Saha
- T8: Opensource tools for automatic speech recognition with Lhotse and Icefall: Training efficient transducers with large data - Fangjun Kuang, Matthew Wiesner, Dr. Piotr Żelasko, Desh Raj, Daniel Povey, Dr. Sanjeev Khudanpur, Dr. Leibny Paola Garcia Perera and Jan "Yenda" Trmal

新一代 Kaldi Job (1/2)

- ▶ We are still in Hiring: Both intern and full-time



新一代 Kaldi Job (2/2)

高级语音算法工程师(J240506014)

北京 / AIoT产品事业部 / 技术 / 1人 / 2024-05-17 更新 /

已结束

职位描述

1. 负责语音相关算法（语音识别/语音增强/语音合成）的研究和模型训练优化工作。
2. 跟踪国内外先进的语音算法技术，推动其研究和业务落地。
3. 其他语音相关算法（声纹、音频分类、VAD）的研究和业务落地。

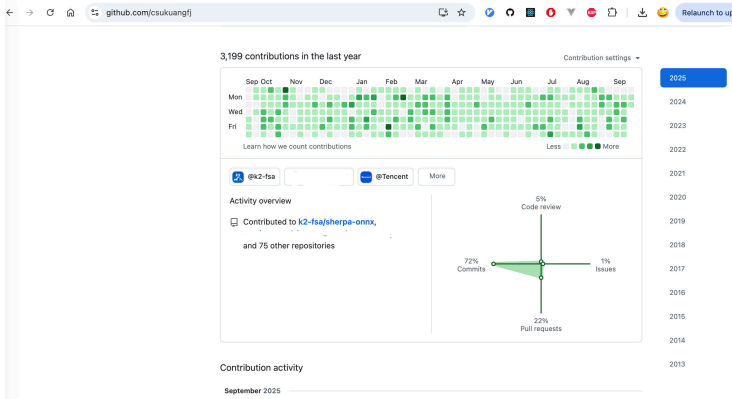
任职要求

- 1) 计算机、人工智能、自动化等相关专业本科或以上学历；
- 2) 熟悉Torch、Tensorflow等深度学习框架，熟练掌握python、C++、shell等编程语言。
- 3) 有语音识别/语音增强/语音合成等其他语音相关算法落地经验者优先。
- 4) 有WeNet/Kaldi/K2等开源语音工具开发经验者优先。
- 5) 在语音技术领域发表过顶会论文或者比赛获奖的优先。

**某公司招聘
职位要求**

个人介绍

- ▶ 中南大学, 本科, 自动化专业
- ▶ 德国斯图加特大学, 硕士, 信息技术专业
- ▶ 现为小米算法工程师, 新一代 Kaldi 团队成员
- ▶ **GitHub** 主页: <https://github.com/csukuangfj>



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

学生需求分析

- ▶ 学习前端知识 (本课程没有涉及)

学生需求分析

- ▶ 学习前端知识（本课程没有涉及）
- ▶ 移动端开发知识（本课程没有涉及）

学生需求分析

- ▶ 学习前端知识（本课程没有涉及）
- ▶ 移动端开发知识（本课程没有涉及）
- ▶ 嵌入式端开发知识（本课程没有涉及）

学生需求分析

- ▶ 学习前端知识（本课程没有涉及）
- ▶ 移动端开发知识（本课程没有涉及）
- ▶ 嵌入式端开发知识（本课程没有涉及）
- ▶ 与实际应用有关，不拘泥于理论
 - ▶ 使用已有的工具
 - ▶ 使用 API，进行二次开发

学生需求分析

- ▶ 学习前端知识（本课程没有涉及）
- ▶ 移动端开发知识（本课程没有涉及）
- ▶ 嵌入式端开发知识（本课程没有涉及）
- ▶ 与实际应用有关，不拘泥于理论
 - ▶ 使用已有的工具
 - ▶ 使用 API，进行二次开发
- ▶ 学习新一代 Kaldi 语音识别技术

基本原则

- ▶ 多一道公式，少一半听众 (尽量做到通俗易懂)

基本原则

- ▶ 多一道公式，少一半听众 (尽量做到通俗易懂)
- ▶ Talk is cheap, show me the code

基本原则

- ▶ 多一道公式，少一半听众 (尽量做到通俗易懂)
- ▶ Talk is cheap, show me the code
- ▶ 可以随时提出问题
 - ▶ 回答问题就有**奖**
 - ▶ 美好本 (10 9 本)
 - ▶ 新++代/Kaldi/新款/V/恤/(4/件)
 - ▶ L+++件./XL/两件./2/XL/++件
 - ▶ 每人限领++件或者++本/(2/选/1)
 - ▶ **数量有限、领完为止**

基本原则

- ▶ 多一道公式，少一半听众 (尽量做到通俗易懂)
- ▶ Talk is cheap, show me the code
- ▶ 可以随时提出问题
 - ▶ 回答问题就有**奖**
 - ▶ 美好本 (10 9 本)
 - ▶ 新++代/Kaldi/新款/V/恤/(4/件)
 - ▶ L+++件./XL/两件./2/XL/++件
 - ▶ 每人限领++件或者++本/(2/选/1)
 - ▶ **数量有限、领完为止**
- ▶ 不需要同学们自己训模型 (鼓励有机器资源的同学动手跑一遍)

教学目的

- ▶ 4 小节课的学习 (9.27, 10.13)
- ▶ 了解语音识别的基本原理
 - ▶ 语音特征的计算
 - ▶ 语音活动检测 (Voice activity detection, VAD)
 - ▶ 3 种端到端语音识别的解码方法
- ▶ 语音识别实战
 - ▶ 新一代 ~~Kaldi~~ 部署框架安装和使用
 - ▶ 电脑端、移动端、嵌入式端、网页端等本地语音识别
 - ▶ 生成字幕实战

现场演示

1. 中文视频和字幕
 - ▶ 雷军年度演讲
 2. 英文视频和字幕
 - ▶ Obama 演讲
 3. 去掉音频中的非人声
 - ▶ 演示 VAD 的作用
- ▶ **备注:** 使用目前开源的 state-of-the-art 模型得到的结果
 - ▶ 本地运行, 不需要访问服务器
 - ▶ 不需要 GPU
 - ▶ 只需个人电脑 CPU

授课目标

- ▶ 现在: **Unknown unknowns** (about speech recognition)

授课目标

- ▶ 现在: **Unknown unknowns** (about speech recognition)
- ▶ 学完这门课后: **Known unknowns** (about speech recognition)

授课目标

- ▶ 现在: **Unknown unknowns** (about speech recognition)
- ▶ 学完这门课后: **Known unknowns** (about speech recognition)
- ▶ 课后自行努力学习后: **Known knowns** (about speech recognition)

来源: <https://www.danielpovey.com/files/Lecture1.pdf>



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

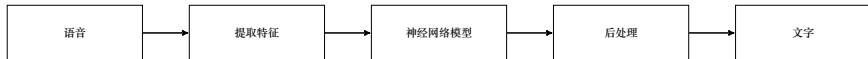
- Transducer

新一代 Kaldi 实战

总结

语音识别基本过程

▶ 端到端 (end-to-end) 语音识别基本流程



▶ 学习神经网络模型的输入和输出

- ▶ 如何从音频变成模型的输入
- ▶ 如何从模型的输出得到文字

▶ 学习 3 种常用的端到端语音识别模型架构

- ▶ CTC
- ▶ Transducer
- ▶ AED (Attention-based Encoder-Decoder)



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

音频

- ▶ 实时采集设备: 麦克风、录音笔、监控摄像头
- ▶ 音视频文件: *.mp3, *.wav, *.m4a, *.opus, *.mp4, *.mov, 等
- ▶ 网络传输: 直播语音流等
- ▶ 绝大多数语音识别模型, 要求音频输入为 wav 格式
 - ▶ 采样率: 16000 Hz
 - ▶ 通道数量: 1
 - ▶ 采样点格式: 16-bit (即 int16_t)

```
$ soxi ./0.wav  
Input File      : './0.wav'  
Channels       : 1  
Sample Rate    : 16000  
Precision      : 16-bit  
Duration       : 00:00:06.62 = 106000 samples ~ 496.875 CDDA sectors  
File Size      : 212k  
Bit Rate       : 256k  
Sample Encoding: 16-bit Signed Integer PCM
```

音频格式转换

- ▶ 多媒体领域的瑞士军刀: **ffmpeg**
- ▶ mp4 视频文件中的音频转成 16k Hz, 单通道, 16-bit 的 wav 格式

```
ffmpeg -i input.mp4 -vn -acodec pcm_s16le -ar 16000 -ac 1 output.wav
```

- ▶ mp3 文件转成 16k Hz, 单通道, 16-bit 的 wav 格式

```
ffmpeg -i input.mp3 -acodec pcm_s16le -ar 16000 -ac 1 output.wav
```

- ▶ 绝大多数常见的音频/视频格式都可以通过 **ffmpeg** 转换为 wav 格式
- ▶ 除了命令行工具, **ffmpeg** 还提供各种编程语言的 API

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

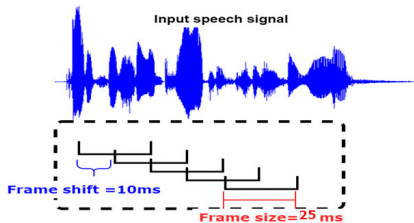
- AED

- Transducer

新一代 Kaldi 实战

总结

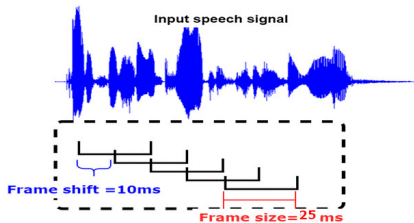
提取特征 — 分帧 (1/2)



► 分帧

- 一般一帧长 25 ms (也叫 frame length), 即 400 个采样点
- 帧偏移 10 ms (也叫 frame shift), 即 160 个采样点

提取特征 — 分帧 (1/2)

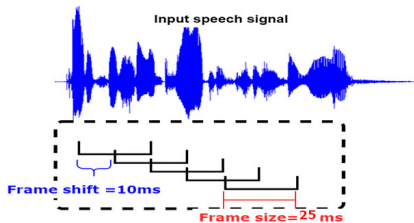


► 分帧

- 一般一帧长 25 ms (也叫 frame length), 即 400 个采样点
- 帧偏移 10 ms (也叫 frame shift), 即 160 个采样点

问题: 1 秒的数据, 可以分成多少帧? (采样率为 16000 Hz)

提取特征 — 分帧 (1/2)



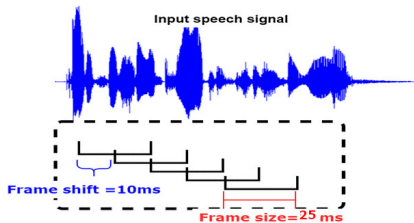
► 分帧

- 一般一帧长 25 ms (也叫 frame length), 即 400 个采样点
- 帧偏移 10 ms (也叫 frame shift), 即 160 个采样点

问题: 1 秒的数据, 可以分成多少帧? (采样率为 16000 Hz)

提示: Conv1d 中, Output shape 与 kernel size 及 stride 的关系 (不考虑 padding)

提取特征 — 分帧 (1/2)



► 分帧

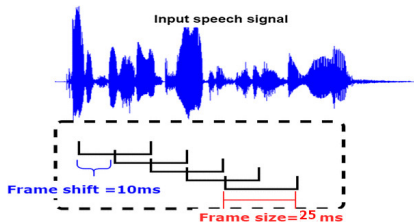
- 一般一帧长 25 ms (也叫 frame length), 即 400 个采样点
- 帧偏移 10 ms (也叫 frame shift), 即 160 个采样点

问题: 1 秒的数据, 可以分成多少帧? (采样率为 16000 Hz)

提示: Conv1d 中, Output shape 与 kernel size 及 stride 的关系 (不考虑 padding)

$$\frac{16000 - 400}{160} + 1 = 98.5, \text{向 0 取整, 得 } 98 \quad (1)$$

提取特征 — 分帧 (1/2)



► 分帧

- 一般一帧长 25 ms (也叫 frame length), 即 400 个采样点
- 帧偏移 10 ms (也叫 frame shift), 即 160 个采样点

问题: 1 秒的数据, 可以分成多少帧? (采样率为 16000 Hz)

提示: Conv1d 中, Output shape 与 kernel size 及 stride 的关系 (不考虑 padding)

$$\frac{16000 - 400}{160} + 1 = 98.5, \text{向 } 0 \text{ 取整, 得 } 98 \quad (1)$$

一般的, 可以近似认为, 1 秒钟 100 帧, 10 秒钟 1000 帧

提取特征 — 分帧 (2/2)

► 实战

```
import numpy as np
np.random.seed(202250927)

sample_rate = 16000
num_seconds = 1
samples = np.random.uniform(low=-1, high=1, size=(sample_rate * num_seconds,))

frame_length_ms = 25
frame_shift_ms = 10

frame_length = int(frame_length_ms * sample_rate / 1000)
frame_shift = int(frame_shift_ms * sample_rate / 1000)

frame_samples_0 = samples[:frame_length]
frame_samples_1 = samples[1*frame_shift:(1*frame_shift + frame_length)]
frame_samples_2 = samples[2*frame_shift:(2*frame_shift + frame_length)]
```

提取特征 — 分帧 (2/2)

► 实战

```
import numpy as np
np.random.seed(202250927)

sample_rate = 16000
num_seconds = 1
samples = np.random.uniform(low=-1, high=1, size=(sample_rate * num_seconds,))

frame_length_ms = 25
frame_shift_ms = 10

frame_length = int(frame_length_ms * sample_rate / 1000)
frame_shift = int(frame_shift_ms * sample_rate / 1000)

frame_samples_0 = samples[:frame_length]
frame_samples_1 = samples[1*frame_shift:(1*frame_shift + frame_length)]
frame_samples_2 = samples[2*frame_shift:(2*frame_shift + frame_length)]
```

问题: 第 n 帧的起始时间是多少?

提取特征 — 分帧 (2/2)

► 实战

```
import numpy as np
np.random.seed(202250927)

sample_rate = 16000
num_seconds = 1
samples = np.random.uniform(low=-1, high=1, size=(sample_rate * num_seconds,))

frame_length_ms = 25
frame_shift_ms = 10

frame_length = int(frame_length_ms * sample_rate / 1000)
frame_shift = int(frame_shift_ms * sample_rate / 1000)

frame_samples_0 = samples[:frame_length]
frame_samples_1 = samples[1*frame_shift:(1*frame_shift + frame_length)]
frame_samples_2 = samples[2*frame_shift:(2*frame_shift + frame_length)]
```

问题: 第 n 帧的起始时间是多少?

$n * \text{frame_shift}$

提取特征 — 分帧 (2/2)

► 实战

```
import numpy as np
np.random.seed(202250927)

sample_rate = 16000
num_seconds = 1
samples = np.random.uniform(low=-1, high=1, size=(sample_rate * num_seconds,))

frame_length_ms = 25
frame_shift_ms = 10

frame_length = int(frame_length_ms * sample_rate / 1000)
frame_shift = int(frame_shift_ms * sample_rate / 1000)

frame_samples_0 = samples[:frame_length]
frame_samples_1 = samples[1*frame_shift:(1*frame_shift + frame_length)]
frame_samples_2 = samples[2*frame_shift:(2*frame_shift + frame_length)]
```

问题: 第 n 帧的起始时间是多少?

$n * \text{frame_shift}$

► 对于一些模型，利用这个信息，可以给识别结果加**时间戳**

提取特征 — 去掉直流偏移

► Remove DC offset

```
def remove_dc_offset(samples):  
    mean = np.mean(samples)  
    return samples - mean
```

提取特征 — 去掉直流偏移

- ▶ Remove DC offset

```
def remove_dc_offset(samples):  
    mean = np.mean(samples)  
    return samples - mean
```

- ▶ 可选的步骤
- ▶ 绝大多数模型都会选择做这个

提取特征 — 预加重

► Preemphasis

```
def preemphasize(samples, coeff=0.97):  
    ans = np.empty_like(samples)  
  
    ans[0] = samples[0] - coeff * samples[0]  
    ans[1:] = samples[1:] - coeff * samples[:-1]  
  
    return ans
```

提取特征 — 预加重

► Preemphasis

```
def preemphasize(samples, coeff=0.97):  
    ans = np.empty_like(samples)  
  
    ans[0] = samples[0] - coeff * samples[0]  
    ans[1:] = samples[1:] - coeff * samples[:-1]  
  
    return ans
```

- 可选的步骤
- 绝大多数模型都会选择做这个

提取特征 — 加窗

- ▶ 有非常多种窗函数
- ▶ hann 窗函数是 PyTorch 里默认的窗函数

▶ https://docs.pytorch.org/docs/stable/generated/torch.hann_window.html

$$w_n = \frac{1}{2} \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right) \quad (2)$$

```
def get_hann_window(n: int):  
    # 请看 https://docs.pytorch.org/docs/stable/generated/torch.hann\_window.html  
    k = np.arange(n)  
    return 0.5 * (1 - np.cos(2 * np.pi * k / (n - 1)))  
  
def apply_window(samples, window):  
    return samples * window
```

提取特征 — 计算 FFT

```
def compute_fft(samples, nfft=512):  
    return np.fft.rfft(samples, nfft)
```

- ▶ 如果采样数是 400 点，后面补 112 个 0，做 512 点的 fft
- ▶ 返回一个复数向量，维度为 $\frac{512}{2} + 1 = 257$

提取特征 — 计算功率谱

```
def compute_power_spectrum(fft_bins):  
    return np.abs(fft_bins) ** 2
```

- ▶ 返回一个**实数**向量，维度为 $\frac{512}{2} + 1 = 257$

提取特征 — 乘以 Mel filter bank 矩阵

```
pip install kaldi-native-fbank
```

```
import kaldi_native_fbank as knf

def get_mel_filter_bank_matrix():
    mel_opts = knf.MelBanksOptions()
    mel_opts.num_bins = 23

    frame_opts = knf.FrameExtractionOptions()
    mel_bank = knf.MelBanks(opts=mel_opts, frame_opts=frame_opts)
    return mel_bank.get_matrix()

matrix = get_mel_filter_bank_matrix()
print(matrix.shape)
```

提取特征 — 乘以 Mel filter bank 矩阵

```
pip install kaldi-native-fbank
```

```
import kaldi_native_fbank as knf

def get_mel_filter_bank_matrix():
    mel_opts = knf.MelBanksOptions()
    mel_opts.num_bins = 23

    frame_opts = knf.FrameExtractionOptions()
    mel_bank = knf.MelBanks(opts=mel_opts, frame_opts=frame_opts)
    return mel_bank.get_matrix()

matrix = get_mel_filter_bank_matrix()
print(matrix.shape)
```

- ▶ 假定特征维度为 23, $n_{\text{fft}} = 512$
- ▶ `matrix` 的形状为 (23, 257)

提取特征 — 乘以 Mel filter bank 矩阵

```
pip install kaldi-native-fbank
```

```
import kaldi_native_fbank as knf

def get_mel_filter_bank_matrix():
    mel_opts = knf.MelBanksOptions()
    mel_opts.num_bins = 23

    frame_opts = knf.FrameExtractionOptions()
    mel_bank = knf.MelBanks(opts=mel_opts, frame_opts=frame_opts)
    return mel_bank.get_matrix()

matrix = get_mel_filter_bank_matrix()
print(matrix.shape)
```

► matrix 有何特点？稀疏性？

提取特征 — All in one

```
def compute_fbank(samples):
    samples = remove_dc_offset(samples)
    samples = preemphasize(samples)

    window = get_hann_window(samples.shape[0])
    samples = samples * window

    fft_bins = compute_fft(samples)
    power_spec = compute_power_spectrum(fft_bins)

    matrix = get_mel_filter_bank_matrix()

    f = np.matmul(matrix, power_spec.reshape(-1, 1)).squeeze(1)

    f = np.where(f == 0, np.finfo(float).eps, f)

    return np.log(f)

feature_frame_0 = compute_fbank(frame_samples_0)
feature_frame_1 = compute_fbank(frame_samples_1)
print(feature_frame_0.shape)
print(feature_frame_1.shape)
```

- ▶ 每一帧的维度都是 23
- ▶ 1 秒钟的特征组成一个二维矩阵, 形状为 (98, 23), 作为神经网络的输入

提取特征 — 实战

- ▶ Google colab notebook:

<https://colab.research.google.com/drive/1NxiyngCA4lDRy5BxqTVDLGDPudWKRwEs?usp=sharing>

- ▶ GitHub 备份: https://github.com/k2-fsa/colab/blob/master/fbank_demo.ipynb



Google Colab



Github 备份

提取特征 — 总结

1. 分帧
2. 去掉直流偏移
3. 预加重
4. 加窗
5. 计算 FFT
6. 计算功率谱
7. 乘以 Mel filter bank 矩阵
8. 取 $\log$

提取特征 — 扩展

- ▶ 1 秒钟的音频，可以得到一个二维矩阵 `features`，形状为 $(98, 23)$
- ▶ **提问**: 计算第 i 帧特征时，是否需要**全部**的 1 秒数据?



Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

在线算法

→ ↺ 🏠 🌐 baike.baidu.com/item/在线算法/4449947 ☆ 🔔

Baidu 百科 在线算法 × 进入词条

专家贡献

在线算法

管理领域术语

🔊 播报 ✎ 编辑 💬 讨论 📺 上传视频 ➕

☆ 收藏

📖 本词条由“科普中国”科学百科词条编写与应用工作项目 审核。

在线算法是指它可以以序列化的方式一个个的处理输入，也就是说在开始时并不需要已经知道所有的输入。

中文名	在线算法	问 题	最短路径
外文名	online algorithm	说 明	时间复杂度 和在线机器学习

► **注意:** 在线算法中的在线，和联网没关系

离线算法

→ ↺ 🏠 🌐 baike.baidu.com/item/离线算法

Baidu 百科 离线算法 × 进入词条 🔊 播报

☆ 收藏 📄 27 🔗 1

离线算法 [lí xiàn suàn fǎ] 🔊 播报 ✎ 编辑 💬 讨论 📺 上传视频 +

基于在执行算法前输入数据已知的基本假设

离线算法(off line algorithms), 是指基于在执行算法前输入数据已知的基本假设, 也就是说, 对于一个离线算法, 在开始时就需要知道问题的所有输入数据, 而且在解决一个问题后就要立即输出结果。

中文名	离线算法	设计策略	在执行算法前输入已知的基本假设
外文名	offline algorithm	前 提	具有问题完全信息

► **注意:** 离线算法中的离线, 和联网没关系

流式语音识别

- ▶ 不需要所有的数据作为输入
- ▶ 使用流式的模型 (在线算法)
- ▶ 边说边识别

非流式语音识别

- ▶ 使用非流式的模型 (离线算法)
- ▶ 需要等待**所有**的数据作为输入, 说完一句话之后, 才开始识别

流式与非流式语音识别比较

表: 流式语音识别与非流式语音识别的对比

对比维度	流式语音识别	非流式语音识别
数据处理方式	边采集边传输边处理, 增量 式处理	需 等待完整 语音数据采集后再处理
延迟特性	低延迟 (通常几百毫秒内), 实时输出中间结果	延迟较高 (依赖整体语音长度), 仅输出最终结果
适用场景	实时对话 (如语音助手、会议字幕)、实时翻译	长语音转写 (如录音文件转文字)、批量处理任务
内存占用	较低 (仅需缓存最近帧数据)	较高 (需加载完整语音数据)
典型应用	微信实时语音转文字、直播字幕生成	录音笔语音转写、长音频归档处理

流式与非流式语音识别比较

表: 流式语音识别与非流式语音识别的对比

对比维度	流式语音识别	非流式语音识别
数据处理方式	边采集边传输边处理, 增量 式处理	需 等待完整 语音数据采集后再处理
延迟特性	低延迟 (通常几百毫秒内), 实时输出中间结果	延迟较高 (依赖整体语音长度), 仅输出最终结果
适用场景	实时对话 (如语音助手、会议字幕)、实时翻译	长语音转写 (如录音文件转文字)、批量处理任务
内存占用	较低 (仅需缓存最近帧数据)	较高 (需加载完整语音数据)
典型应用	微信实时语音转文字、直播字幕生成	录音笔语音转写、长音频归档处理

1. **问题** 刷榜 (刷准确率排名) 一般用流式还是非流式识别?

流式与非流式语音识别比较

表: 流式语音识别与非流式语音识别的对比

对比维度	流式语音识别	非流式语音识别
数据处理方式	边采集边传输边处理, 增量 式处理	需 等待完整 语音数据采集后再处理
延迟特性	低延迟 (通常几百毫秒内), 实时输出中间结果	延迟较高 (依赖整体语音长度), 仅输出最终结果
适用场景	实时对话 (如语音助手、会议字幕)、实时翻译	长语音转写 (如录音文件转文字)、批量处理任务
内存占用	较低 (仅需缓存最近帧数据)	较高 (需加载完整语音数据)
典型应用	微信实时语音转文字、直播字幕生成	录音笔语音转写、长音频归档处理

1. **问题** 刷榜 (刷准确率排名) 一般用流式还是非流式识别?
2. **问题** 流式语音识别可否识别一个文件?

流式与非流式语音识别比较

表: 流式语音识别与非流式语音识别的对比

对比维度	流式语音识别	非流式语音识别
数据处理方式	边采集边传输边处理, 增量 式处理	需 等待完整 语音数据采集后再处理
延迟特性	低延迟 (通常几百毫秒内), 实时输出中间结果	延迟较高 (依赖整体语音长度), 仅输出最终结果
适用场景	实时对话 (如语音助手、会议字幕)、实时翻译	长语音转写 (如录音文件转文字)、批量处理任务
内存占用	较低 (仅需缓存最近帧数据)	较高 (需加载完整语音数据)
典型应用	微信实时语音转文字、直播字幕生成	录音笔语音转写、长音频归档处理

1. **问题** 刷榜 (刷准确率排名) 一般用流式还是非流式识别?
2. **问题** 流式语音识别可否识别一个文件?
3. **问题** 非流式语音识别对一个超长文件进行识别时, 如何避免 **out of memory** 错误
4. **问题** 非流式语音识别可否/如何和麦克风实时录音一起用?

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

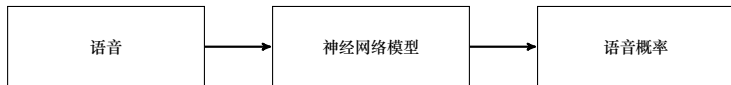
- Transducer

新一代 Kaldi 实战

总结

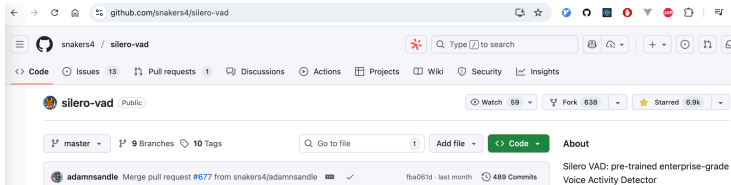
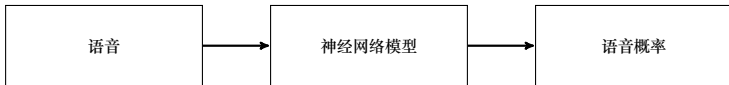
语音活动检测 (Voice activity detection, VAD)

- ▶ 基于神经网络模型的 VAD
- ▶ 模型文件大小：几百 KB



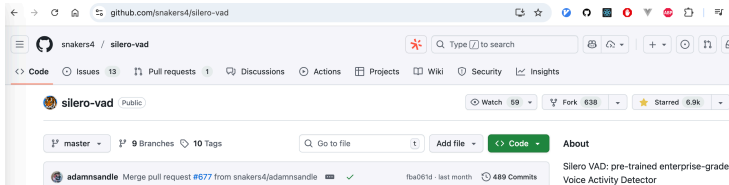
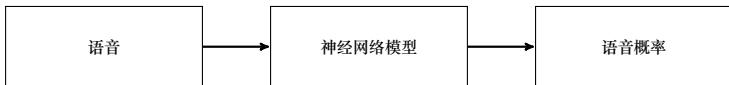
语音活动检测 (Voice activity detection, VAD)

- ▶ 基于神经网络模型的 VAD
- ▶ 模型文件大小：几百 KB



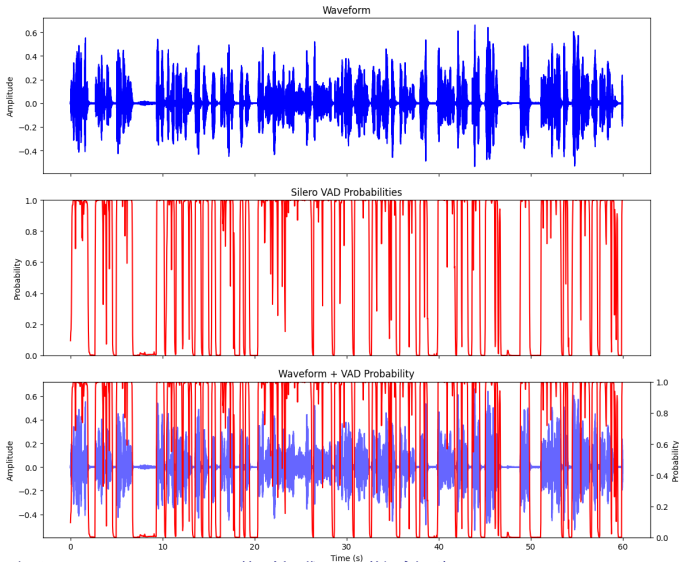
语音活动检测 (Voice activity detection, VAD)

- ▶ 基于神经网络模型的 VAD
- ▶ 模型文件大小: 几百 KB

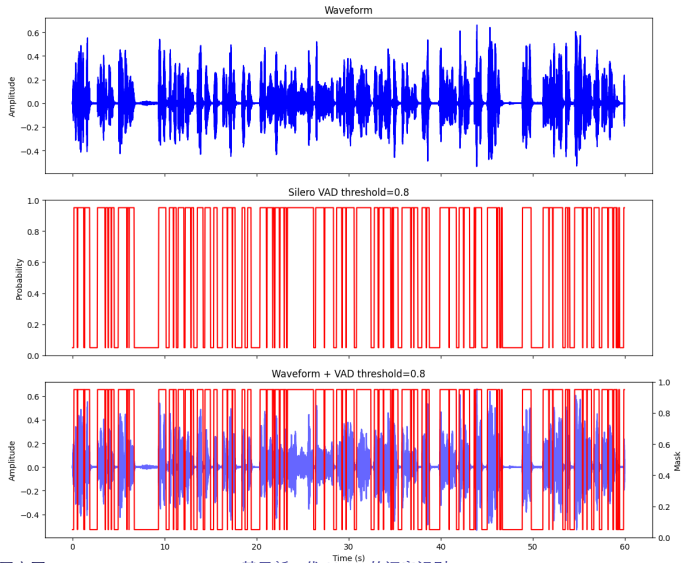


- ▶ 输入 **32 ms** 的数据, 模型输出一个数, 代表这 32 ms 数据包含语音的**概率**
- ▶ frame length 32 ms, frame shift 也为 32 ms
- ▶ 即语音帧之间没有重叠 (无 overlap)

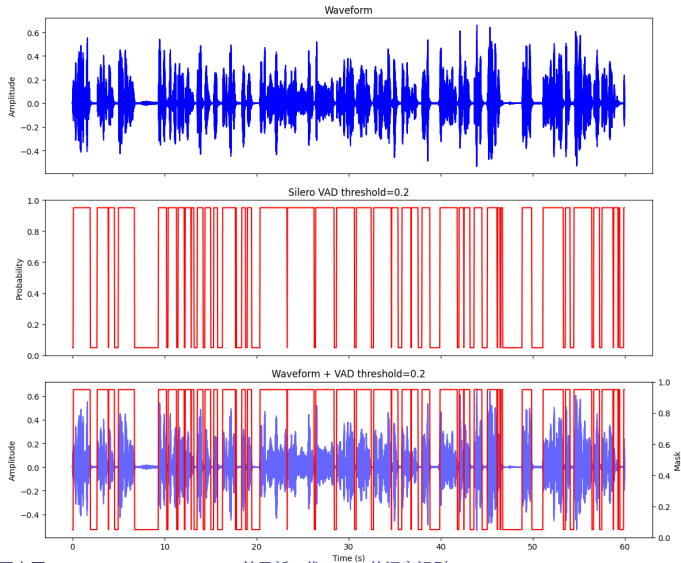
VAD 模型输出演示 (1/3)



VAD 模型输出演示 (2/3)



VAD 模型输出演示 (2/3)



VAD 代码实战

- ▶ Google colab notebook:

https://colab.research.google.com/drive/1DH-MVrPKYxjqtQe9cny0pdBLG_wZlh03?usp=sharing

- ▶ GitHub 备份: https://github.com/k2-fsa/colab/blob/master/vad_demo.ipynb



Google Colab



GitHub 备份

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?
5. **问题** 如何从输出的概率中, 得到语音片段?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?
5. **问题** 如何从输出的概率中, 得到语音片段?
6. **问题** 如何得到语音片段在原始音频中的**起始时间**?

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?
5. **问题** 如何从输出的概率中, 得到语音片段?
6. **问题** 如何得到语音片段在原始音频中的**起始时间**?
7. **问题** 如何处理来自麦克风的**实时**音频?
 - ▶ 即如何处理**流式**输入

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?
5. **问题** 如何从输出的概率中, 得到语音片段?
6. **问题** 如何得到语音片段在原始音频中的**起始时间**?
7. **问题** 如何处理来自麦克风的**实时**音频?
 - ▶ 即如何处理**流式**输入
8. **问题** 如何在浏览器里运行 VAD (结合麦克风)?
 - ▶ 需要解决哪些问题

VAD — 思考

1. **问题** silero vad 属于流式还是非流式模型?
2. **问题** silero vad 能否区分人声和动物的声音?
3. **问题** silero vad 能否区分人声和背景音乐?
4. **问题** 如何处理**短暂**的**停顿**?
5. **问题** 如何从输出的概率中, 得到语音片段?
6. **问题** 如何得到语音片段在原始音频中的**起始时间**?
7. **问题** 如何处理来自麦克风的**实时**音频?
 - ▶ 即如何处理**流式**输入
8. **问题** 如何在浏览器里运行 VAD (结合麦克风)?
 - ▶ 需要解决哪些问题
9. **问题** 如何在 Android 手机上运行 VAD (结合麦克风)?
 - ▶ 需要解决哪些问题

VAD 效果演示

- ▶ 网页版 <https://modelscope.cn/studios/csukuangfj/web-assembly-vad-sherpa-onnx>
- ▶ Android APP <https://k2-fsa.github.io/sherpa/onnx/vad/apk-cn.html>



网页版演示



Android版演示

VAD 效果演示的相关代码

- ▶ 网页版: <https://github.com/k2-fsa/sherpa-onnx/tree/master/wasm/vad>
 - ▶ WebAssembly
 - ▶ JavaScript
 - ▶ HTML
- ▶ Android 版: <https://github.com/k2-fsa/sherpa-onnx/tree/master/android/SherpaOnnxVad>
 - ▶ Kotlin
 - ▶ Jetpack Compose
- ▶ 底层都是 C++ 代码
 - ▶ 环形缓冲
□ <https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/csrc/circular-buffer.h>
 - ▶ VAD 模型
□ <https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/csrc/silero-vad-model.h>
 - ▶ VAD 入口 <https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/csrc/voice-activity-detector.h>

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

Contents

关于我们

课程介绍

语音识别

语音识别基本过程

音频

提取特征

离线与在线算法

语音活动检测 (VAD)

3 种端到端语音识别架构的解码

CTC

AED

Transducer

新一代 Kaldi 实战

总结

CTC (1/7)

- ▶ https://www.cs.toronto.edu/~graves/icml_2006.pdf
- ▶ **2006** 年提出

Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks

Alex Graves¹
Santiago Fernández¹
Faustino Gomez¹
Jürgen Schmidhuber^{1,2}

ALEX@IDSIA.CH
SANTIAGO@IDSIA.CH
TINO@IDSIA.CH
JUERGEN@IDSIA.CH

¹ Istituto Dalle Molle di Studi sull'Intelligenza Artificiale (IDSIA), Galleria 2, 6928 Manno-Lugano, Switzerland

² Technische Universität München (TUM), Boltzmannstr. 3, 85748 Garching, Munich, Germany

CTC (1/7)

- ▶ https://www.cs.toronto.edu/~graves/icml_2006.pdf
- ▶ **2006** 年提出

Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks

Alex Graves¹
Santiago Fernández¹
Faustino Gomez¹
Jürgen Schmidhuber^{1,2}

ALEX@IDSIA.CH
SANTIAGO@IDSIA.CH
TINO@IDSIA.CH
JUERGEN@IDSIA.CH

¹ Istituto Dalle Molle di Studi sull'Intelligenza Artificiale (IDSIA), Galleria 2, 6928 Manno-Lugano, Switzerland

² Technische Universität München (TUM), Boltzmannstr. 3, 85748 Garching, Munich, Germany

- ▶ 3 种端到端语音识别架构中，有 2 种是一作 Alex Graves 提出来的
- ▶ 四作 Jürgen Schmidhuber 也是 LSTM 的作者

CTC (2/7)

- ▶ **假设** 我们只有 a, c, t 三个字母
- ▶ 以字母为建模单元。称建模单元为 **token**
- ▶ cat.wav 这个音频里面对应的语音是 cat 的发音

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
logits = model(features)
```

- ▶ logits 是一个 2-d 矩阵，维度是 (num_out_frames, vocab_size)
- ▶ 假设 num_out_frames 为 5
- ▶ **问题**: 如何从 logits 这个矩阵，得到文字 cat

CTC (3/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (num_out_frames, vocab_size)
- ▶ 假设 num_out_frames 为 5
- ▶ **问题**: 如何从 logits 这个矩阵，得到文字 cat
- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 要解决的**问题**
 - ▶ cat 只对应 3 个 token，即 c, a, t. 如何从输出的 5 个 token 中，变成 3 个 token，即 c, a, t
 - ▶ 音频中的静音帧如何处理，对应哪个 token

CTC (3/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (num_out_frames, vocab_size)
- ▶ 假设 num_out_frames 为 5
- ▶ **问题**: 如何从 logits 这个矩阵，得到文字 cat
- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 要解决的**问题**
 - ▶ cat 只对应 3 个 token，即 c, a, t. 如何从输出的 5 个 token 中，变成 3 个 token，即 c, a, t
 - ▶ 音频中的静音帧如何处理，对应哪个 token
- ▶ 引入一个新的 token，叫做 ϵ ，用来处理静音帧

CTC (3/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (num_out_frames, vocab_size)
- ▶ 假设 num_out_frames 为 5
- ▶ **问题**: 如何从 logits 这个矩阵，得到文字 cat
- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 要解决的**问题**
 - ▶ cat 只对应 3 个 token，即 c, a, t. 如何从输出的 5 个 token 中，变成 3 个 token，即 c, a, t
 - ▶ 音频中的静音帧如何处理，对应哪个 token
- ▶ 引入一个新的 token，叫做 ϵ ，用来处理静音帧
- ▶ 所以 vocab_size 为 4，即 ϵ , c, a, t

CTC (3/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (num_out_frames, vocab_size)
- ▶ 假设 num_out_frames 为 5
- ▶ **问题**: 如何从 logits 这个矩阵，得到文字 cat
- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 要解决的**问题**
 - ▶ cat 只对应 3 个 token，即 c, a, t. 如何从输出的 5 个 token 中，变成 3 个 token，即 c, a, t
 - ▶ 音频中的静音帧如何处理，对应哪个 token
- ▶ 引入一个新的 token，叫做 ϵ ，用来处理静音帧
- ▶ 所以 vocab_size 为 4，即 ϵ , c, a, t
- ▶ 给每个 token 编号

编号	0	1	2	3
token	ϵ	a	c	t

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一些可能的输出

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)
- ▶ 给每个 token 编号

编号	0	1	2	3
token	ε	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一些可能的输出
- ▶ 00213, 即 ε ε c a t
 - ▶ 第 0 帧输出 token ID 0
 - ▶ 第 1 帧输出 token ID 0
 - ▶ 第 2 帧输出 token ID 2
 - ▶ 第 3 帧输出 token ID 1
 - ▶ 第 4 帧输出 token ID 3

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一些可能的输出
- ▶ 00213, 即 € € c a t
- ▶ 02013, 即 € c € a t

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一些可能的输出
- ▶ 00213, 即 € € c a t
- ▶ 02013, 即 € c € a t
- ▶ 20130, 即 c € a t €

CTC (4/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一些可能的输出
- ▶ 00213, 即 € € c a t
- ▶ 02013, 即 € c € a t
- ▶ 20130, 即 c € a t €
- ▶ 去掉 €, 把所有 token 依次连起来，就是识别结果

CTC (5/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一个 token 可能持续好几帧

CTC (5/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一个 token 可能持续好几帧
- ▶ 一些可能的输出

CTC (5/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)
- ▶ 给每个 token 编号

编号	0	1	2	3
token	ε	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一个 token 可能持续好几帧
- ▶ 一些可能的输出
- ▶ 02213, 即 ε c c a t
 - ▶ 第 0 帧输出 token ID 0
 - ▶ 第 1 帧输出 token ID 2
 - ▶ 第 2 帧输出 token ID 2
 - ▶ 第 3 帧输出 token ID 1
 - ▶ 第 4 帧输出 token ID 3

CTC (5/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一个 token 可能持续好几帧
- ▶ 一些可能的输出
- ▶ 02213, 即 € c c a t
- ▶ 21113, 即 c a a a t

CTC (5/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)

- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

- ▶ CTC 要求每个输出帧，都要对应一个 token。所以本例中从 logits 需要产生 5 个 token
- ▶ 一个 token 可能持续好几帧
- ▶ 一些可能的输出
- ▶ 02213, 即 € c c a t
- ▶ 21113, 即 c a a a t
- ▶ 把相邻重复的 token 去重, 然后去掉 €, 把所有 token 依次连起来, 就是识别结果

CTC (6/7)

- ▶ logits 是一个 2-d 矩阵，维度是 (5, 4)
- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

```
ids = logits.argmax(axis=-1)
```

1. 对 ids 去重
2. 去 €
3. 查表, 即 token ID 转成 token
4. 把所有 token 依次连起来, 即为识别结果

CTC (7/7)

- ▶ logits 是一个 2-d 矩阵, 维度是 (5, 4)
- ▶ 给每个 token 编号

编号	0	1	2	3
token	€	a	c	t

```
ids = logits.argmax(axis=-1)
```

- ▶ CTC 特点
 - ▶ 输出帧之间, 互不影响, 相互**独立**
 - ▶ **并行**解码
 - ▶ 输出帧数**不能少于** 识别文本对应的 token 数量
 - ▶ 对于 cat 这个单词对应的音频
 - ▶ cat 由 3 个 token 组成
 - ▶ 输出帧, 至少要包含 3 帧

常用的 CTC 模型

1. 阿里巴巴开源的 SenseVoice 模型:

<https://github.com/FunAudioLLM/SenseVoice>

- ▶ 支持中英日韩粤 5 种语言
- ▶ 速度快、准确率高

2. 西北工业大学开源的 SenseVoice 粤语模型:

https://huggingface.co/ASLP-lab/WSYue-ASR/tree/main/u2pp_conformer_yue

3. 中国电信人工智能研究院开源的 TeleSpeech-ASR 模型:

<https://github.com/Tele-AI/TeleSpeech-ASR>

- ▶ 支持 8 种方言

4. DataoceanAI 和 清华大学联合开源的 Dolphin 模型:

<https://github.com/DataoceanAI/Dolphin>

- ▶ 支持 40 种东亚语言
- ▶ 支持 22 种中国方言

CTC 解码实战

- ▶ Google colab notebook:

<https://colab.research.google.com/drive/16psxsWi7Z6TTrvZ1V4IOXt4jAZXWGbbs?usp=sharing>

- ▶ GitHub 备份: https://github.com/k2-fsa/colab/blob/master/ctc_decoding_demo.ipynb



Google Colab



GitHub 备份

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

AED 介绍

- ▶ **A**ttention-based **E**ncoder-**D**ecoder 架构
- ▶ 很有代表性的一个模型是 **OpenAI** 开源的 **Whisper** 模型
- ▶ <http://github.com/openai/whisper>
- ▶ **bos**: beginning of sentence
- ▶ **eos**: end of sentence

第 1 步，运行 encoder 模型

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
encoder_out = run_encoder(features)
```

第 2 步，运行 decoder 模型

```
ids = []  
state = get_init_state()  
current = bos  
  
while True:  
    logits, state = run_decoder(current, state, encoder_out)  
    current = logits.argmax().item()  
    if current == eos:  
        break  
  
    ids.append(current)  
  
// 查表: 把 ids 变成 token  
// 然后把所有 token 依次连接起来
```

AED 解码分析

第 1 步, 运行 encoder 模型

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
encoder_out = run_encoder(features)
```

第 2 步, 运行 decoder 模型

```
ids = []  
state = get_init_state()  
current = bos  
  
while True:  
    logits, state = run_decoder(current, state, encoder_out)  
    current = logits.argmax().item()  
    if current == eos:  
        break  
  
    ids.append(current)  
  
// 查表: 把 ids 变成 token  
// 然后把所有 token 依次连接起来
```

- ▶ 自回归 (类似 RNN/LSTM/GRU)
- ▶ 不容易并行解码

AED 解码分析

第 1 步, 运行 encoder 模型

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
encoder_out = run_encoder(features)
```

第 2 步, 运行 decoder 模型

```
ids = []  
state = get_init_state()  
current = bos  
  
while True:  
    logits, state = run_decoder(current, state, encoder_out)  
    current = logits.argmax().item()  
    if current == eos:  
        break  
  
    ids.append(current)  
  
// 查表: 把 ids 变成 token  
// 然后把所有 token 依次连接起来
```

- ▶ 自回归 (类似 RNN/LSTM/GRU)
- ▶ 不容易并行解码
- ▶ **问题:** 如果一直不出现 **eos**, 该怎么办?

AED 解码例子

第 1 步，运行 encoder 模型

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
encoder_out = run_encoder(features)
```

- ▶ 以 cat.wav 为例子
- ▶ 输入 bos，输出 c
- ▶ 输入 c，输出 a
- ▶ 输入 a，输出 t
- ▶ 输入 t，输出 eos

第 2 步，运行 decoder 模型

```
ids = [ ]  
state = get_init_state()  
current = bos  
  
while True:  
    logits, state = run_decoder(current, state, encoder_out)  
    current = logits.argmax().item()  
    if current == eos:  
        break  
  
    ids.append(current)  
  
// 查表：把 ids 变成 token  
// 然后把所有 token 依次连接起来
```

常用的 AED 模型

1. **OpenAI** 开源的 Whisper 模型:
<https://github.com/openai/whisper>
2. 小红书开源的 FireRedASR 模型:
<https://github.com/FireRedTeam/FireRedASR>
 - ▶ 70k 小时训练数据
 - ▶ 支持中英文
3. 英伟达开源的 Canary 模型:
<https://huggingface.co/nvidia/canary-180m-flash>

AED 解码实战

- ▶ Google colab notebook:

<https://colab.research.google.com/drive/1XA69PowRkkZUAhFJ8sHL6k7MEWfpiqrJ?usp=sharing>

- ▶ GitHub 备份: https://github.com/k2-fsa/colab/blob/master/aed_decoding_demo.ipynb



Google Colab



GitHub 备份

Contents

关于我们

课程介绍

语音识别

语音识别基本过程

音频

提取特征

离线与在线算法

语音活动检测 (VAD)

3 种端到端语音识别架构的解码

CTC

AED

Transducer

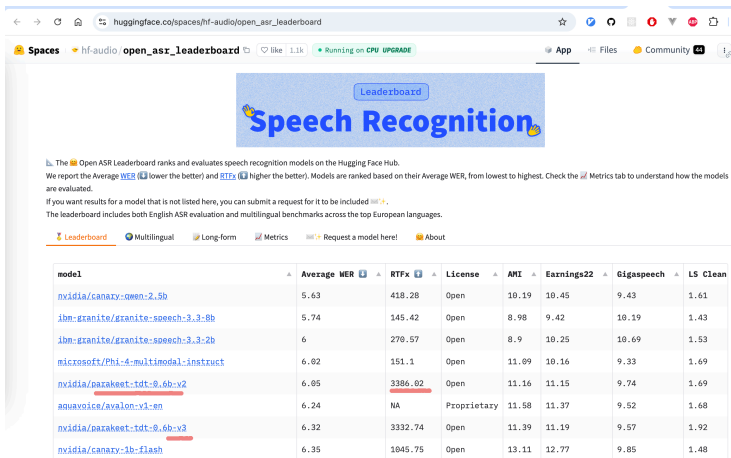
新一代 Kaldi 实战

总结

Transducer 介绍

▶ 很有代表性的一个模型是 英伟达开源的 **parakeet-tdt-0.6b-v2** 模型

▶ <https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2>



The screenshot shows the Hugging Face Open ASR Leaderboard. At the top, there's a banner for "Speech Recognition". Below it, a description states: "The Open ASR Leaderboard ranks and evaluates speech recognition models on the Hugging Face Hub. We report the Average WER (lower the better) and RTFx (higher the better). Models are ranked based on their Average WER, from lowest to highest. Check the Metrics tab to understand how the models are evaluated. If you want results for a model that is not listed here, you can submit a request for it to be included. The leaderboard includes both English ASR evaluation and multilingual benchmarks across the top European languages."

Navigation tabs include: Leaderboard, Multilingual, Long-form, Metrics, Request a model here!, and About.

model	Average WER	RTFx	License	AMI	Earnings22	Gigaspeech	LS Clean
nvidia/canary-qwen-2.5b	5.63	418.28	Open	10.19	10.45	9.43	1.61
ibm-granite/granite-speech-3.3-8b	5.74	145.42	Open	8.98	9.42	10.19	1.43
ibm-granite/granite-speech-3.3-2b	6	270.57	Open	8.9	10.25	10.69	1.53
microsoft/Phi-4-multimodal-instruct	6.02	151.1	Open	11.09	10.16	9.33	1.69
nvidia/parakeet-tdt-0.6b-v2	6.05	3386.02	Open	11.16	11.15	9.74	1.69
aquavoice/avalon-v1-en	6.24	NA	Proprietary	11.58	11.37	9.52	1.68
nvidia/parakeet-tdt-0.6b-v3	6.32	3332.74	Open	11.39	11.19	9.57	1.92
nvidia/canary-1b-flash	6.35	1045.75	Open	13.11	12.77	9.85	1.48

Transducer 组成 (1/3)

- ▶ 3 部分: encoder 模型, decoder 模型, joiner 模型
- ▶ **encoder**

```
samples = load("cat.wav")  
  
features = compute_features(samples)  
  
encoder_out = encoder(features)
```

- ▶ encoder_out 的 shape 为 (num_frames, encoder_dim)
- ▶ 一般把这里的 num_frames 叫做 T

Transducer 组成 (2/3)

- ▶ 3 部分: encoder 模型, decoder 模型, joiner 模型
- ▶ **decoder**

```
current = xx  
decoder_out = decoder(current)
```

- ▶ `current` 为当前识别出来的 token ID
 - ▶ 开始识别之前的 ID 为一个特殊的 blank ID
 - ▶ AED 里面有 bos 和 eos
 - ▶ transducer 里面有 blank
- ▶ `decoder_out` 的 shape 为 $(1, \text{decoder_dim})$

Transducer 组成 (3/3)

- ▶ 3 部分: encoder 模型, decoder 模型, joiner 模型
- ▶ **joiner**

```
encoder_out_t = encoder_out[t:t+1]  
  
logits = joiner(encoder_out_t, decoder_out)
```

- ▶ t 的取值从 0 到 $T-1$
- ▶ T 为 `encoder_out.shape[0]`

Transducer 解码

- ▶ 3 部分: encoder 模型, decoder 模型, joiner 模型

```
ids = [ ]
t = 0
while t < T:
    encoder_out_t = encoder_out[t:t+1]
    logits = joiner(encoder_out_t, decoder_out)
    current = logits.argmax()
    if current == blank:
        t += 1
    else:
        ids.append(current)

    decoder_out = decoder(current)

// 查表: 把 ids 变成 token
// 然后把所有 token 依次连接起来
```

Transducer 解码

- ▶ 3 部分: encoder 模型, decoder 模型, joiner 模型

```
ids = [ ]
t = 0
while t < T:
    encoder_out_t = encoder_out[t:t+1]
    logits = joiner(encoder_out_t, decoder_out)
    current = logits.argmax()
    if current == blank:
        t += 1
    else:
        ids.append(current)

    decoder_out = decoder(current)

// 查表: 把 ids 变成 token
// 然后把所有 token 依次连接起来
```

- ▶ **问题** 如果一直不解码出来 blank, 该怎么办?

常用的 transducer 模型

1. 很有代表性的一个模型是 英伟达开源的 **parakeet-tdt-0.6b-v2** 和 **parakeet-tdt-0.6b-v3**
2. 新一代 **Kaldi** 开源的 Zipformer transducer 模型

transducer 解码实战

- ▶ Google colab notebook:

https://colab.research.google.com/drive/1l8EqNGTsdmxbG6uu1mDCBH4QpI_KKaP-?usp=sharing

- ▶ GitHub 备份: https://github.com/k2-fsa/colab/blob/master/transducer_decoding_demo.ipynb



Google Colab



GitHub 备份

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

新一代 Kaldi 实战

- ▶ 代码: <http://github.com/k2-fsa/sherpa-onnx>
- ▶ 文档: <https://k2-fsa.github.io/sherpa/onnx/index.html>

Supported functions

Speech recognition	Speech synthesis	Source separation
✓	✓	✓

Speaker identification	Speaker diarization	Speaker verification
✓	✓	✓

Spoken Language identification	Audio tagging	Voice activity detection
✓	✓	✓

Keyword spotting	Add punctuation	Speech enhancement
✓	✓	✓

新一代 Kaldi 实战

- ▶ 代码: <http://github.com/k2-fsa/sherpa-onnx>
- ▶ 文档: <https://k2-fsa.github.io/sherpa/onnx/index.html>

Supported platforms

Architecture	Android	iOS	Windows	macOS	linux	HarmonyOS
x64	✓		✓	✓	✓	✓
x86	✓		✓			
arm64	✓	✓	✓	✓	✓	✓
arm32	✓				✓	✓
riscv64					✓	

生成字幕实战

- ▶ <https://github.com/k2-fsa/sherpa-onnx/blob/master/python-api-examples/generate-subtitles.py>

```
./python-api-examples/generate-subtitles.py \  
--silero-vad-model=./silero_vad.onnx \  
--tokens=./sherpa-onnx-sense-voice-zh-en-ja-ko-yue-2024-07-17/tokens.txt \  
--sense-voice=./sherpa-onnx-sense-voice-zh-en-ja-ko-yue-2024-07-17/model.int8.onnx \  
/Users/fangjun/asr-course/demo-1-subtitles/lei-jun.wav
```

Contents

关于我们

课程介绍

语音识别

- 语音识别基本过程

- 音频

- 提取特征

- 离线与在线算法

- 语音活动检测 (VAD)

3 种端到端语音识别架构的解码

- CTC

- AED

- Transducer

新一代 Kaldi 实战

总结

总结

- ▶ 提取语音特征
- ▶ VAD
- ▶ 3 种端到端语音识别模型架构的解码
 1. CTC
 2. AED
 3. transducer
- ▶ 如何用新一代 Kaldi 生成字幕

Thank you!